

Interfaces

What is an interface?

Why is an interface useful?

An interface is a classlike construct that **contains only constants and abstract methods**. In many ways, an interface is similar to an abstract class, but the intent of an interface is **to specify common behavior for objects**. For example, you can specify that the objects are comparable, edible, cloneable using appropriate interfaces.

Define an Interface

To distinguish an interface from a class, Java uses the following syntax to define an interface:

```
public interface InterfaceName {  
    constant declarations;  
    abstract method signatures;  
}
```

Example:

```
public interface Edible {  
    /** Describe how to eat */  
    public abstract String howToEat();  
}
```

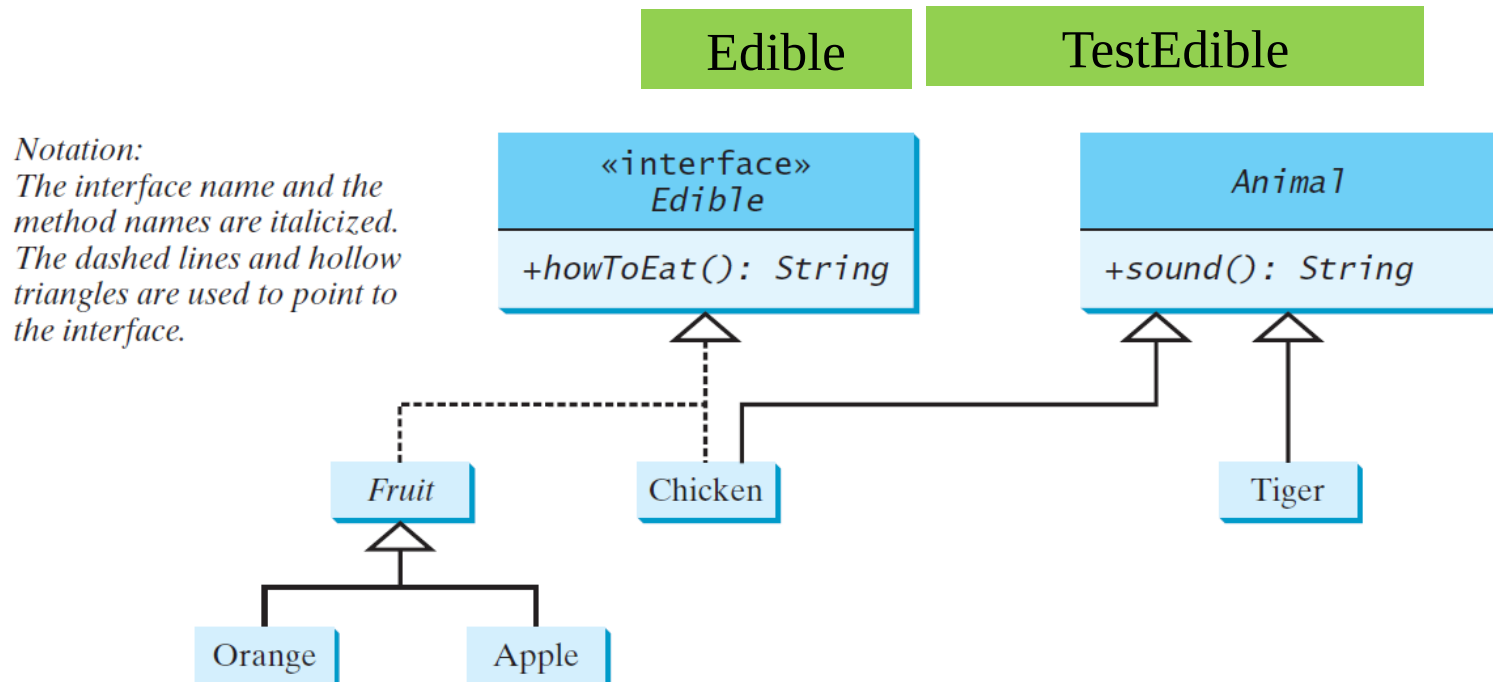
Interface is a Special Class

An interface is treated like a special class in Java:

- ❑ Each interface is compiled into a separate bytecode file, just like a regular class.
- ❑ Like an abstract class, you cannot create an instance from an interface using the new operator,
- ❑ You can use an interface more or less the same way you use an abstract class. For example, you can use an interface as a data type for a variable, as the result of casting, and so on.

Example

You can now use the Edible interface to specify whether an object is edible. This is accomplished by letting the class for the object implement this interface using the implements keyword. For example, the classes Chicken and Fruit implement the Edible interface (See TestEdible).



Omitting Modifiers in Interfaces

All data fields are *public final static* and all methods are *public abstract* in an interface. For this reason, these modifiers can be omitted, as shown below:

```
public interface T1 {  
    public static final int K = 1;  
  
    public abstract void p();  
}
```

Equivalent

```
public interface T1 {  
    int K = 1;  
  
    void p();  
}
```

A constant defined in an interface can be accessed using syntax `InterfaceName.CONSTANT_NAME` (e.g., `T1.K`).

Example: The Comparable Interface

```
// This interface is defined in
// java.lang package
package java.lang;

public interface Comparable<E> {
    public int compareTo(E o);
}
```

The toString, equals, and hashCode Methods

Each wrapper class:

- Overrides the toString, equals, and hashCode methods defined in the Object class.
- Implement the Comparable interface, the compareTo method is implemented in these classes.

Integer and BigInteger Classes

```
public class Integer extends Number
    implements Comparable<Integer> {
    // class body omitted

    @Override
    public int compareTo(Integer o) {
        // Implementation omitted
    }
}
```

```
public class BigInteger extends Number
    implements Comparable<BigInteger> {
    // class body omitted

    @Override
    public int compareTo(BigInteger o) {
        // Implementation omitted
    }
}
```

String and Date Classes

```
public class String extends Object
    implements Comparable<String> {
    // class body omitted

    @Override
    public int compareTo(String o) {
        // Implementation omitted
    }
}
```

```
public class Date extends Object
    implements Comparable<Date> {
    // class body omitted

    @Override
    public int compareTo(Date o) {
        // Implementation omitted
    }
}
```

Generic `sort` Method

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object. All the following expressions are **true**.

```
n instanceof Integer  
n instanceof Object  
n instanceof Comparable
```

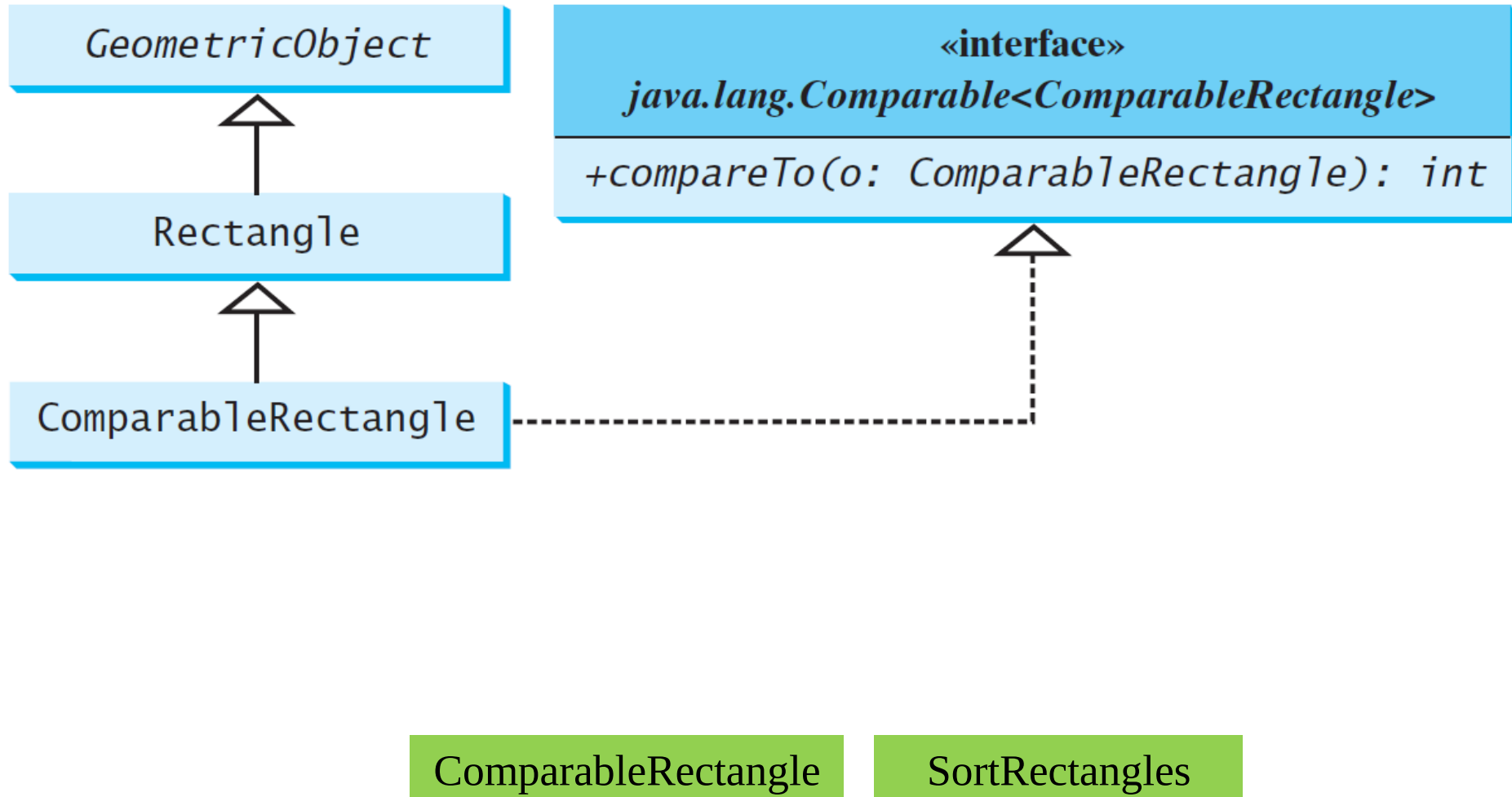
```
s instanceof String  
s instanceof Object  
s instanceof Comparable
```

```
d instanceof java.util.Date  
d instanceof Object  
d instanceof Comparable
```

The `java.util.Arrays.sort(array)` method requires that the elements in an array are instances of `Comparable<E>`.

SortComparableObjects

Defining Classes to Implement Comparable



Interfaces vs. Abstract Classes

In an interface, the data must be constants; an abstract class can have all types of data.

Each method in an interface has only a signature without implementation; an abstract class can have concrete methods.

	<i>Variables</i>	<i>Constructors</i>	<i>Methods</i>
Abstract class	No restrictions.	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator.	No restrictions.
Interface	All variables must be public static final .	No constructors. An interface cannot be instantiated using the new operator.	All methods must be public abstract instance methods

Java allows only single inheritance for class extension but allows multiple extensions for interfaces. For example,

```
public class NewClass extends BaseClass implements  
Interface1, ... , InterfaceN {  
  
... }
```

Interfaces vs. Abstract Classes

An interface can inherit other interfaces using the extends keyword. Such an interface is called a subinterface. For example,

```
public interface NewInterface extends Interface1, ... ,  
InterfaceN {  
  
    // constants and abstract methods  
  
}
```

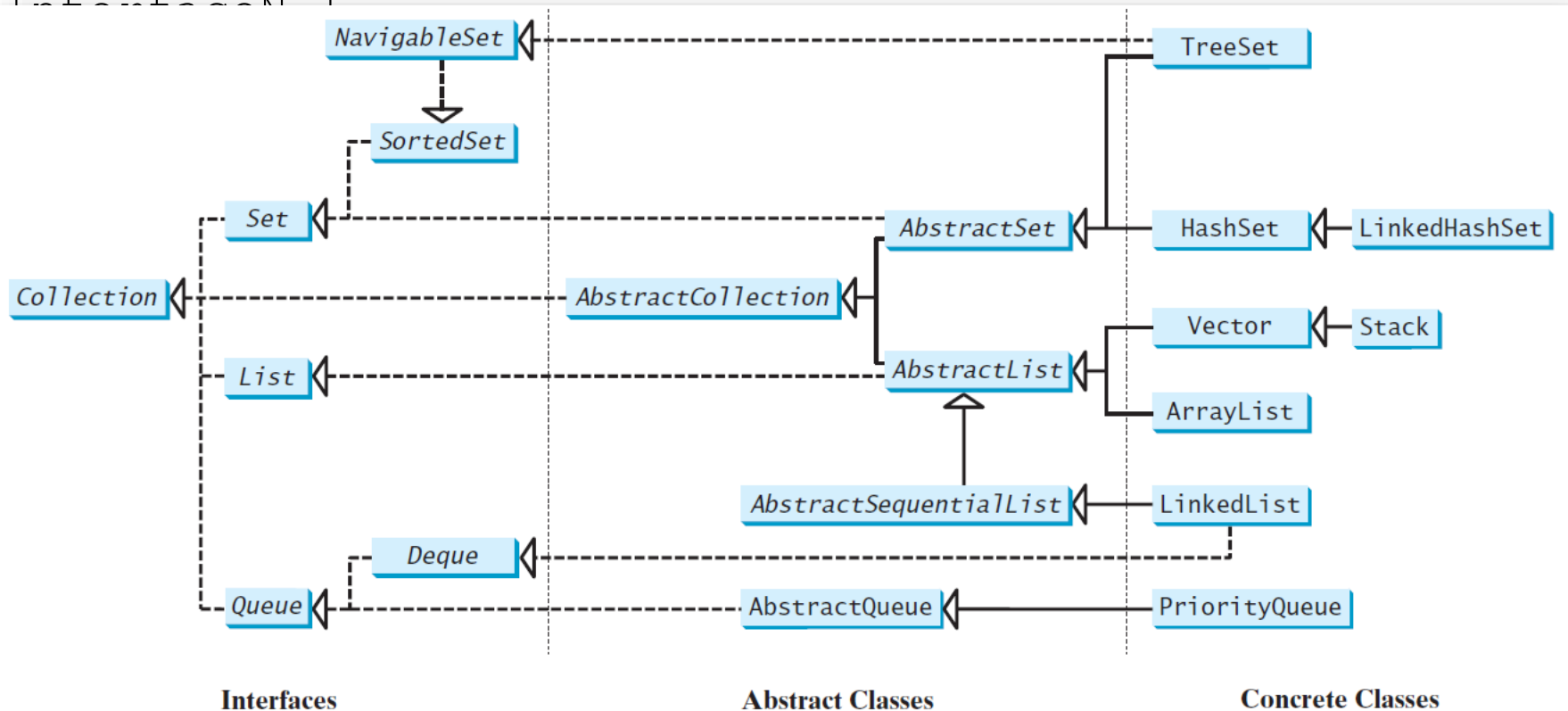
A class implementing NewInterface must implement the abstract methods defined in NewInterface, Interface1, . . . , and InterfaceN.

An interface can extend other interfaces, but not classes.

Interfaces vs. Abstract Classes

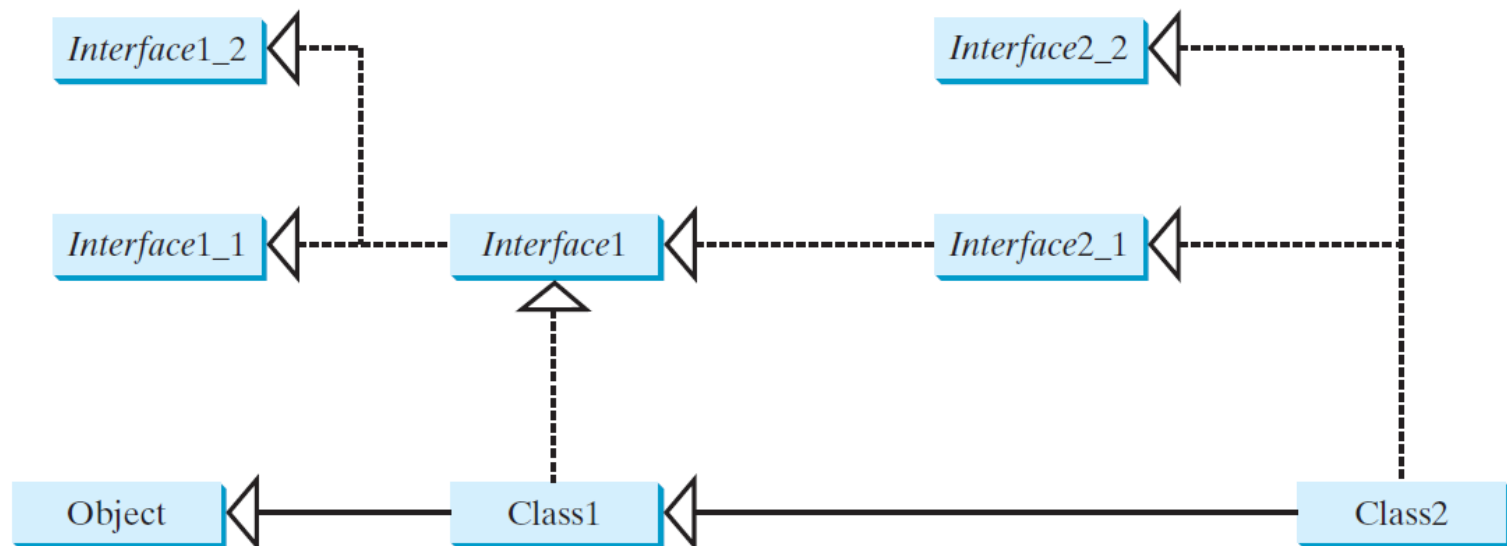
An interface can inherit other interfaces using the extends keyword. Such an interface is called a subinterface. For example,

```
public interface NewInterface extends Interface1, ... ,  
InterfaceN {
```



Interfaces vs. Abstract Classes, cont.

All classes share a single root, the `Object` class, but there is no single root for interfaces. Like a class, an interface also defines a type. A variable of an interface type can reference any instance of the class that implements the interface. If a class extends an interface, this interface plays the same role as a superclass. You can use an interface as a data type and cast a variable of an interface type to its subclass, and vice versa.



Suppose that `c` is an instance of `Class2`. `c` is also an instance of `Object`, `Class1`, `Interface1`, `Interface1_1`, `Interface1_2`, `Interface2_1`, and `Interface2_2`.

Caution: conflict interfaces

In rare occasions, a class may implement two interfaces with conflict information. e.g.:

- two same constants with different values
- or two methods with same signature but different return type.

This type of errors will be detected by the compiler.

Whether to use an interface or a class?

How do you decide whether to use an interface or a class?

- A strong is-a relationship that clearly describes a parent-child relationship should be modeled using classes. For example, a staff member is a person. A weak is-a relationship, also known as an is-kind-of relationship, indicates that an object possesses a certain property. A weak is-a relationship can be modeled using interfaces. For example, all strings are comparable, so the String class implements the Comparable interface.
- You can also use interfaces to circumvent single inheritance restriction if multiple inheritance is desired. In the case of multiple inheritance, you have to design one as a superclass, and others as interface.

Exception Handling

Motivations

When a program runs into a runtime error, the program terminates abnormally. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.

Exception-Handling Overview

Show runtime error

Quotient

Fix it using an if statement

QuotientWithIf

With a method

QuotientWithMethod

Exception Advantages

QuotientWithException

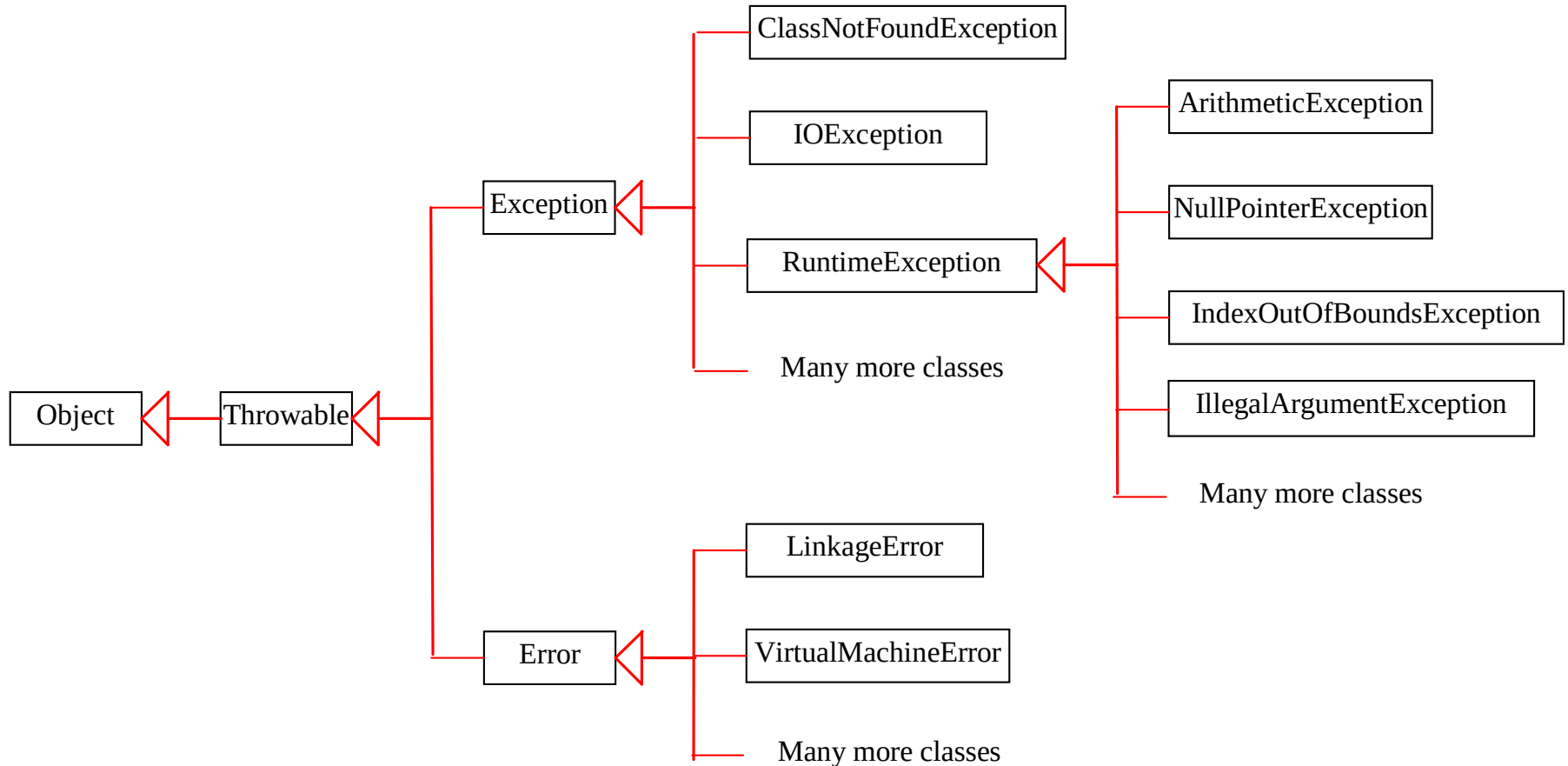
Now you see the *advantages* of using exception handling. It enables a method to throw an exception to its caller. Without this capability, a method must handle the exception or terminate the program.

Handling InputMismatchException

InputMismatchExceptionDemo

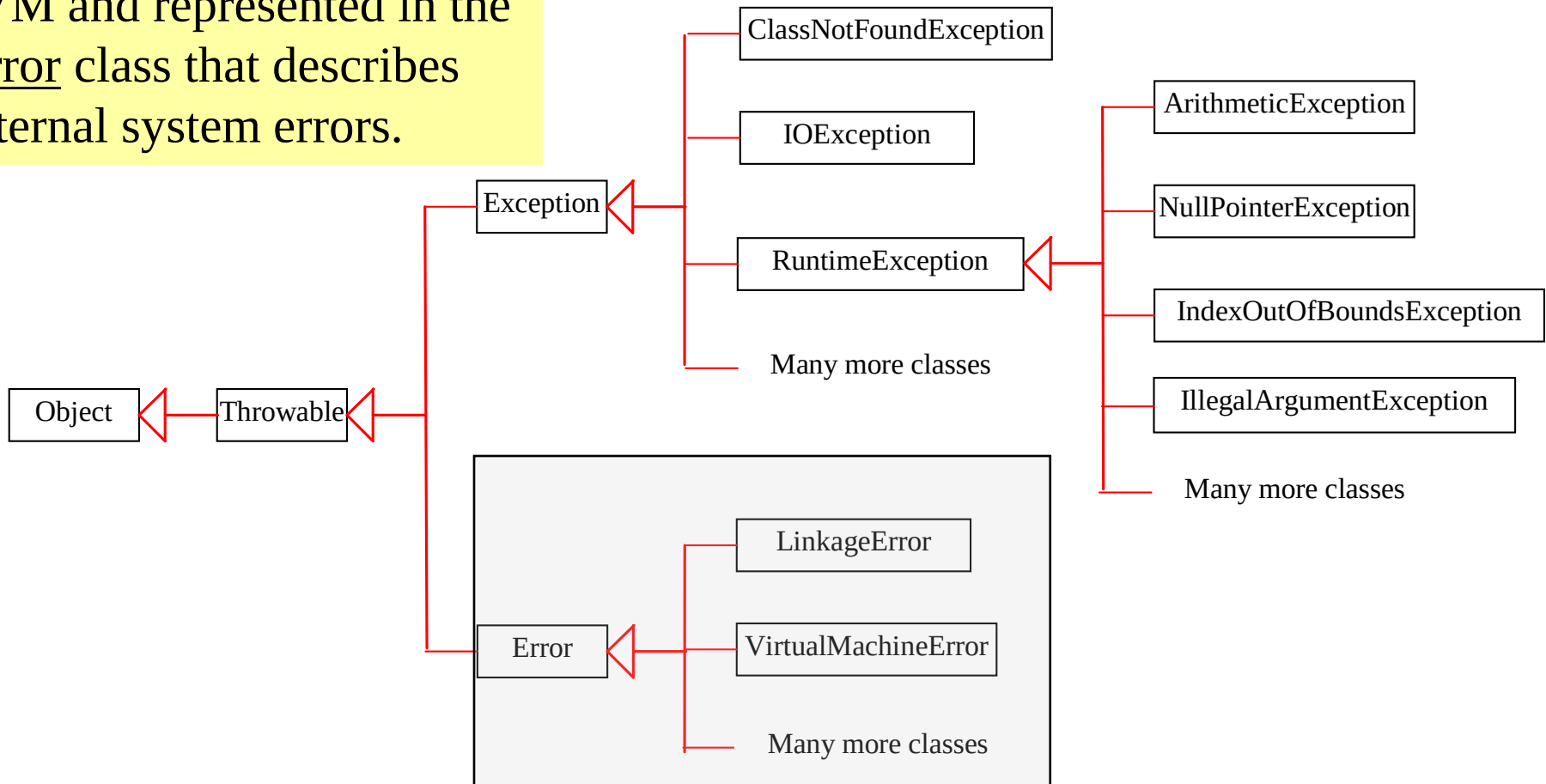
By handling InputMismatchException, your program will continuously read an input until it is correct.

Exception Types



System Errors

System errors are thrown by JVM and represented in the Error class that describes internal system errors.

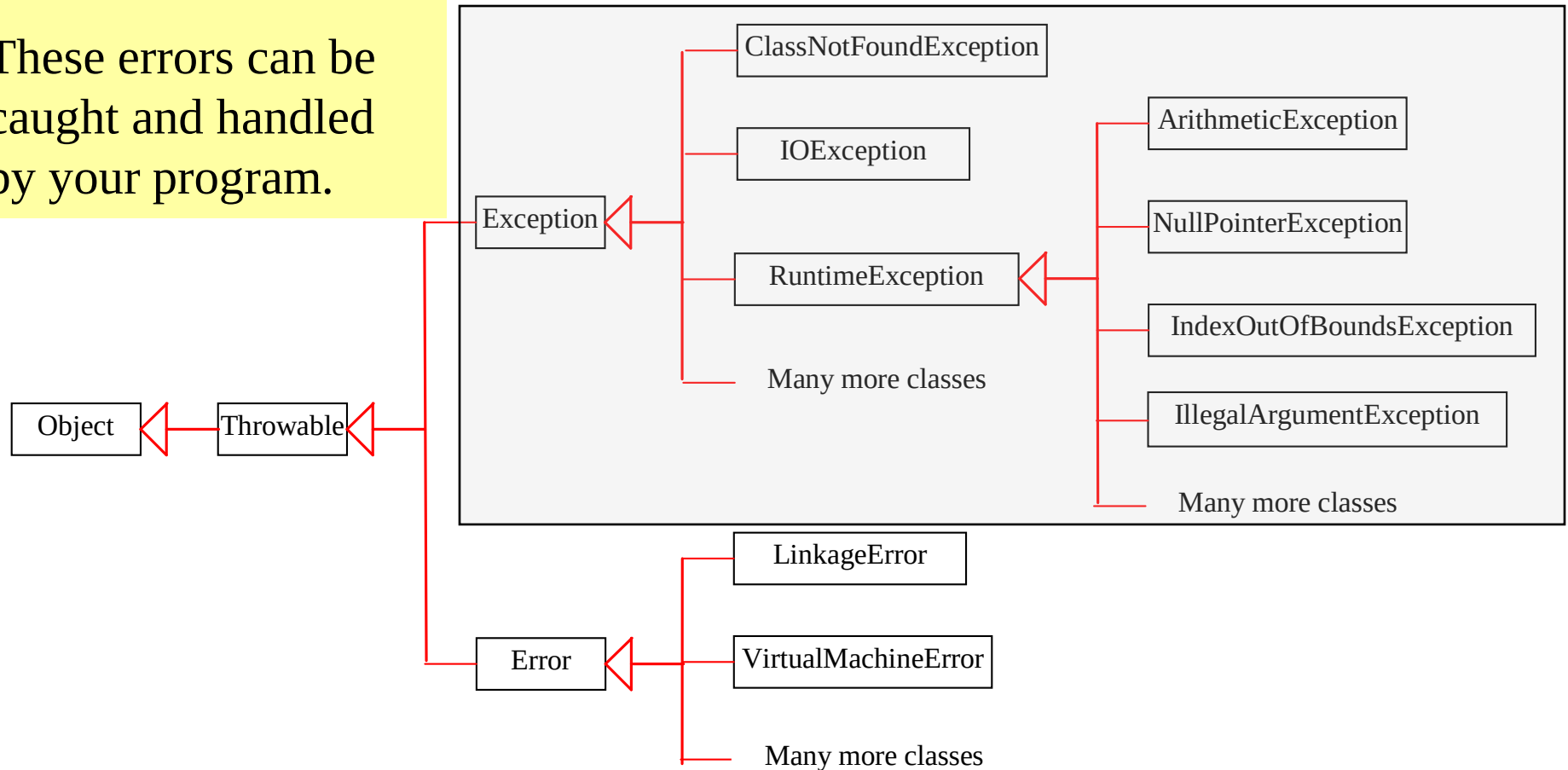


Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

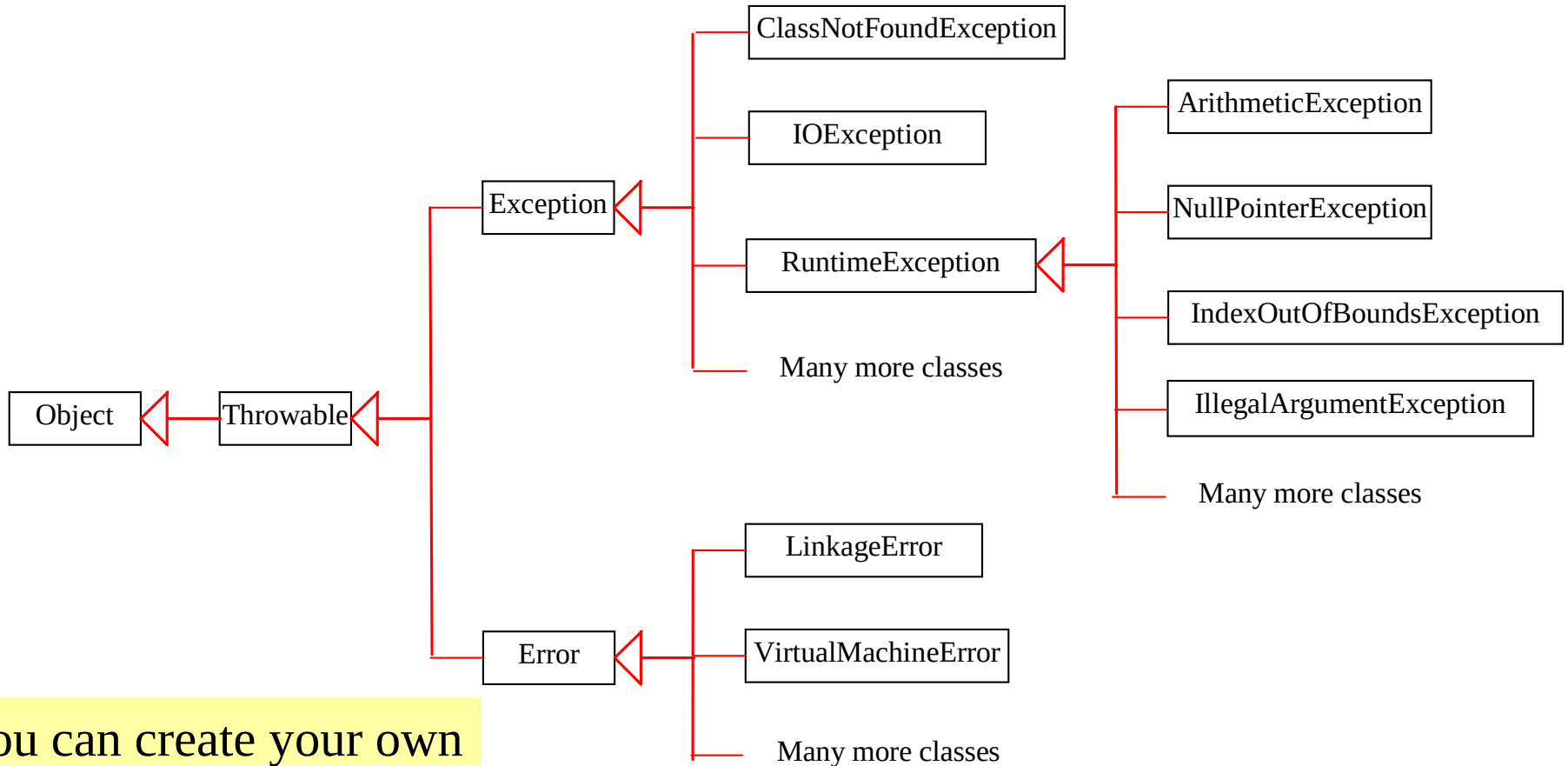
Exception describes errors caused by your program and external circumstances.

These errors can be caught and handled by your program.

Exceptions

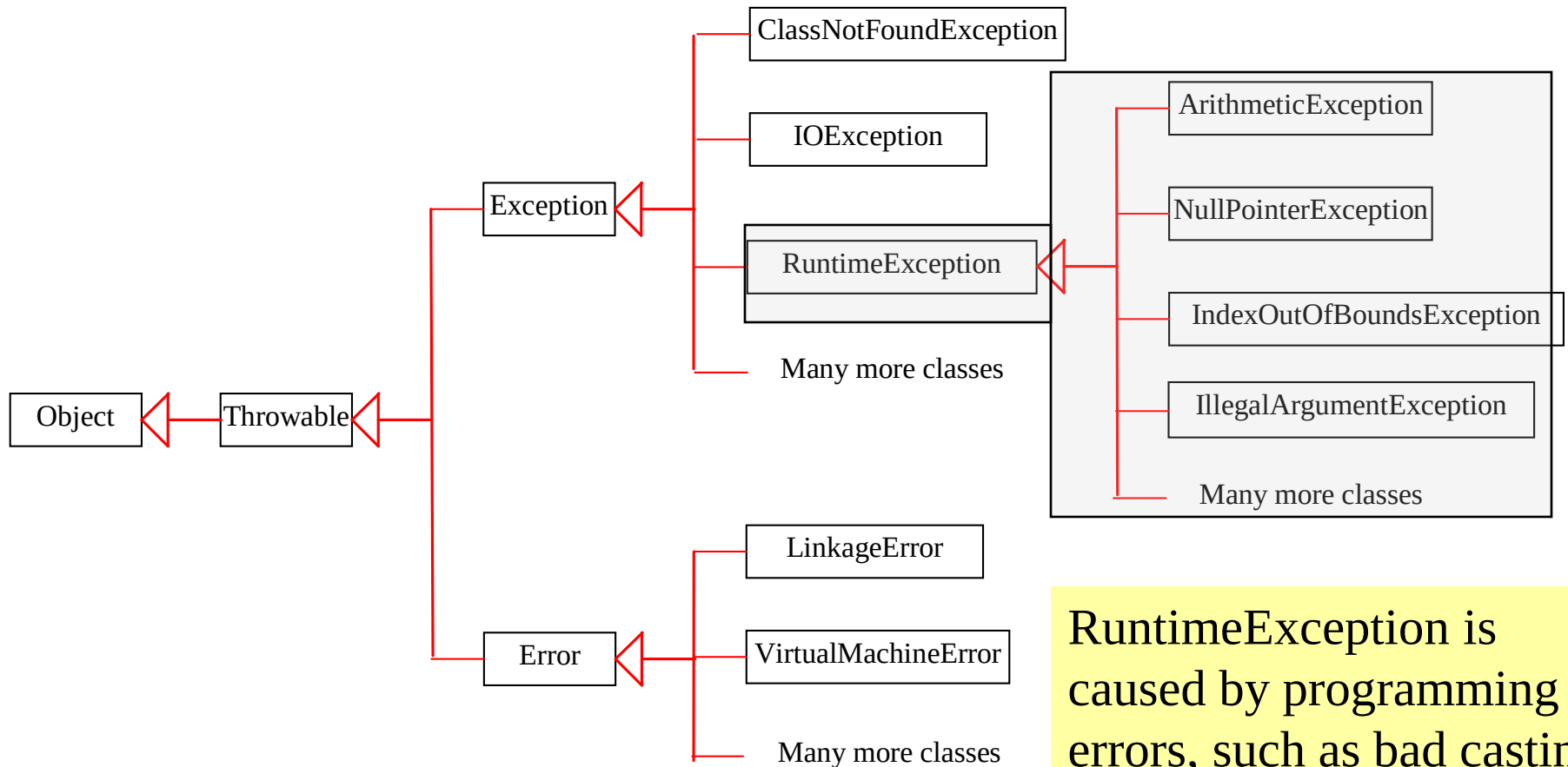


Exceptions



You can create your own exception classes by extending **Exception** or a subclass of **Exception**.

Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Checked Exceptions vs. Unchecked Exceptions

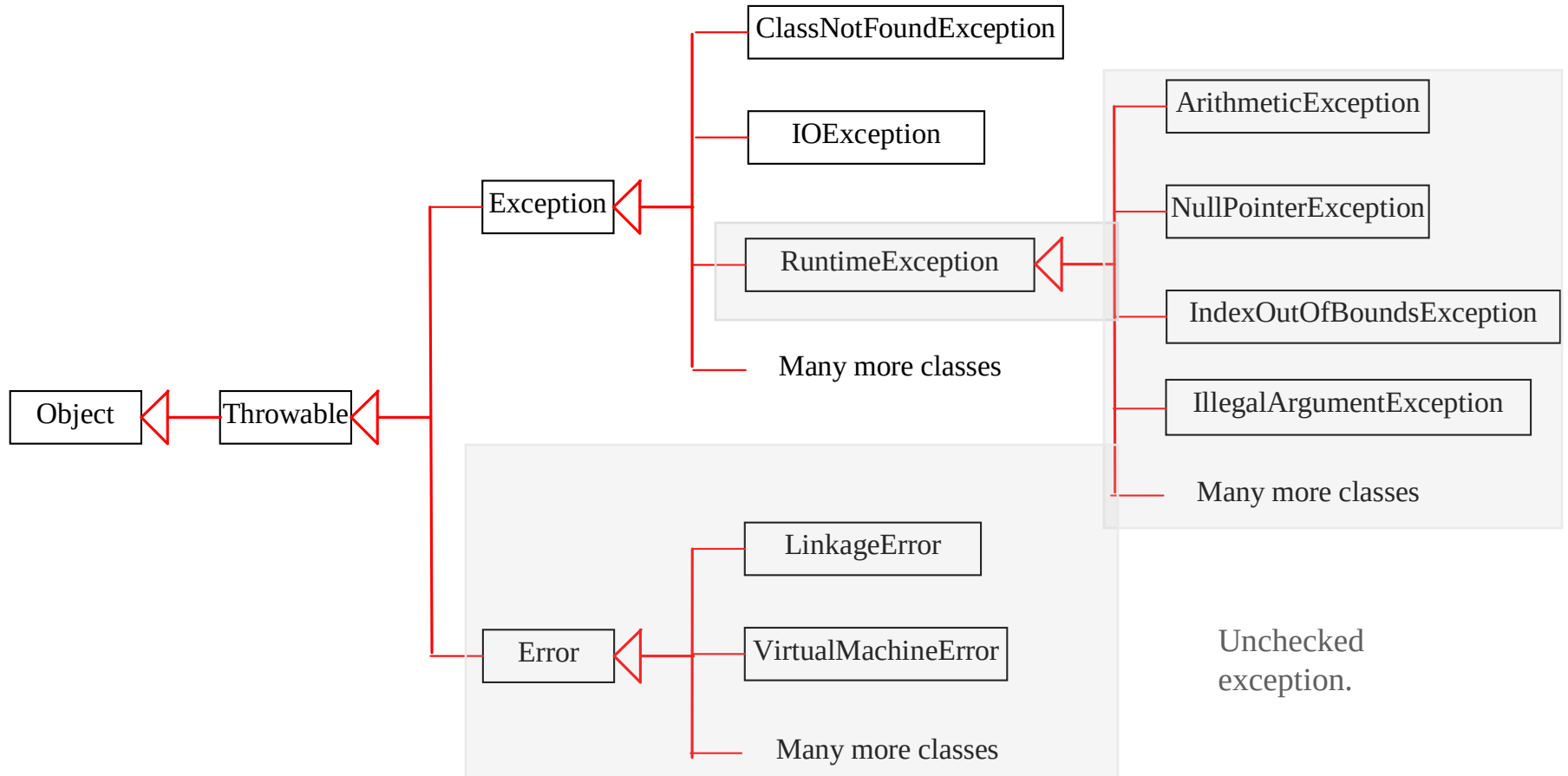
RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

Unchecked Exceptions

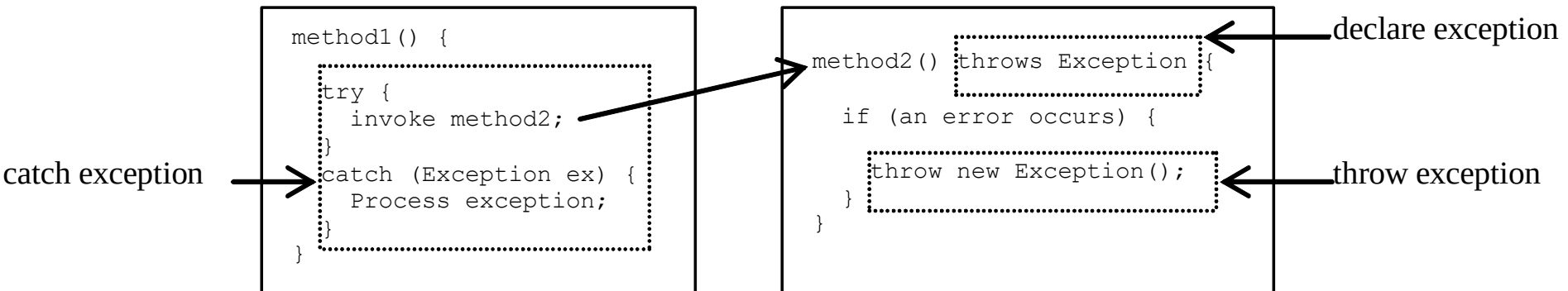
In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a `NullPointerException` is thrown if you access an object through a reference variable before an object is assigned to it; an `IndexOutOfBoundsException` is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in the program.

To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

Unchecked Exceptions



Declaring, Throwing, and Catching Exceptions



Declaring Exceptions

Every method must state the types of **checked exceptions** it might throw. This is known as *declaring exceptions*.

```
public void myMethod()  
    throws IOException
```

```
public void myMethod()  
    throws IOException, OtherException
```


Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();  
throw ex;
```

Throwing Exceptions Example

```
/** Set a new radius */  
public void setRadius(double newRadius)  
    throws IllegalArgumentException {  
    if (newRadius >= 0)  
        radius = newRadius;  
    else  
        throw new IllegalArgumentException(  
            "Radius cannot be negative");  
}
```

Catching Exceptions

```
try {  
    statements;    // Statements that may throw exceptions  
}  
catch (Exception1 exVar1) {  
    handler for exception1;  
}  
catch (Exception2 exVar2) {  
    handler for exception2;  
}  
...  
catch (ExceptionN exVar3) {  
    handler for exceptionN;  
}
```

Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception, you must invoke it in a try-catch block or declare to throw the exception in the calling method.

Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {  
    if (a file does not exist) {  
        throw new IOException("File does not exist");  
    }  
  
    ...  
}
```

And method p1 invokes method p2, you have to write the code as shown in (a) or (b).

```
void p1() {  
    try {  
        p2();  
    }  
    catch (IOException ex) {  
        ...  
    }  
}
```

(a)

```
void p1() throws IOException {  
  
    p2();  
  
}
```

(b)

Example: Declaring, Throwing, and Catching Exceptions

- Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the setRadius method in the Circle class defined in Chapter 9. The new setRadius method throws an exception if radius is negative.

CircleWithException

TestCircleWithException

Rethrowing Exceptions

```
try {  
    statements;  
}  
catch (TheException ex) {  
    perform operations before exits;  
    throw ex;  
}
```

The `finally` Clause

You may want some code to be executed regardless of whether an exception occurs or is caught. Java has a **finally** clause that can be used to accomplish this objective.

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The code in the **finally** block is executed under all circumstances.

The `finally` Clause

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Trace a Program Execution

Suppose no
exceptions in the
statements

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;

Trace a Program Execution

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The final block is
always executed

Next statement;

Trace a Program Execution

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;

Next statement in the
method is executed

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Suppose an exception
of type Exception1 is
thrown in statement2

Next statement;

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The exception is handled.

Next statement;

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}  
  
Next statement;
```

The final block is
always executed.

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;

The next statement in the method is now executed.

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
catch (Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

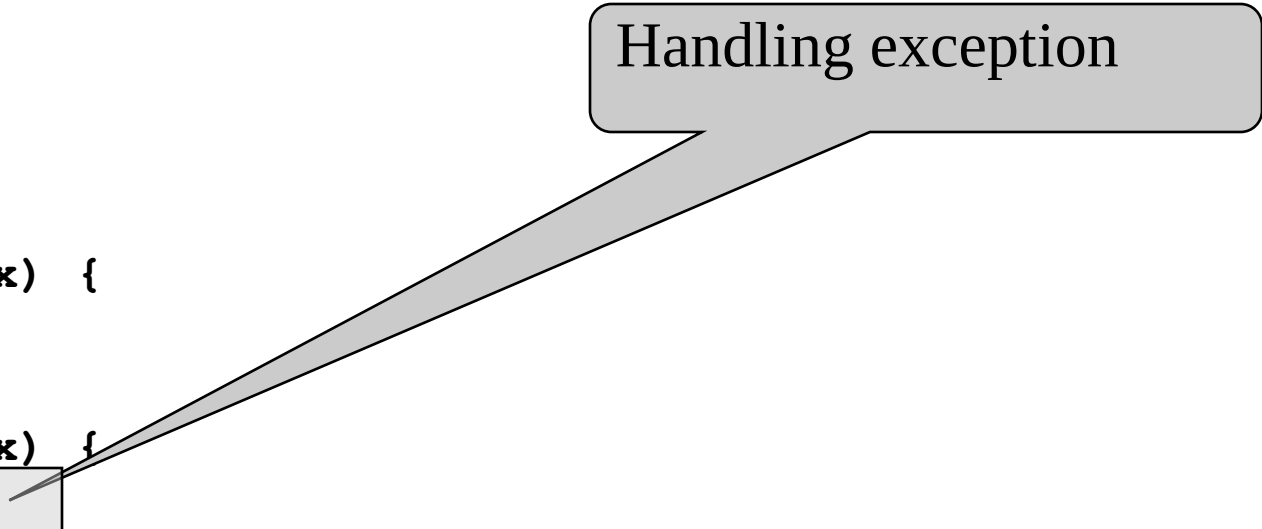
statement2 throws an exception of type Exception2.

Next statement;

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
catch (Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Handling exception

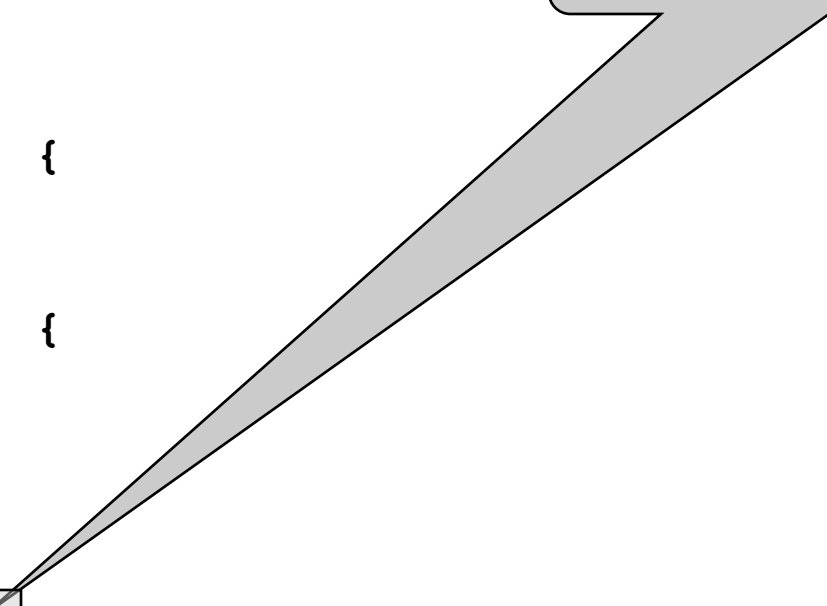


Next statement;

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Execute the final block

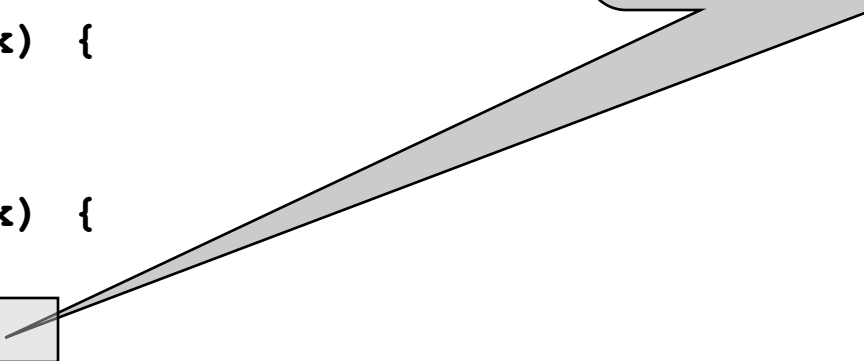


Next statement;

Trace a Program Execution

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
catch (Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;



Rethrow the exception
and control is
transferred to the caller

Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify. Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

When to Throw Exceptions

- An exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

When to Use Exceptions

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {  
    System.out.println(refVar.toString());  
}  
catch (NullPointerException ex) {  
    System.out.println("refVar is null");  
}
```

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
```

```
    System.out.println(refVar.toString());
```

```
else
```

```
    System.out.println("refVar is null");
```


Defining Custom Exception Classes

- ❑ Use the exception classes in the API whenever possible.
- ❑ Define custom exception classes if the predefined classes are not sufficient.
- ❑ Define custom exception classes by extending Exception or a subclass of Exception.

Custom Exception Class Example

In Listing 13.8, the setRadius method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler, you have to create a custom exception class.

InvalidRadiusException

CircleWithRadiusException

TestCircleWithRadiusException

Text IO

The File Class

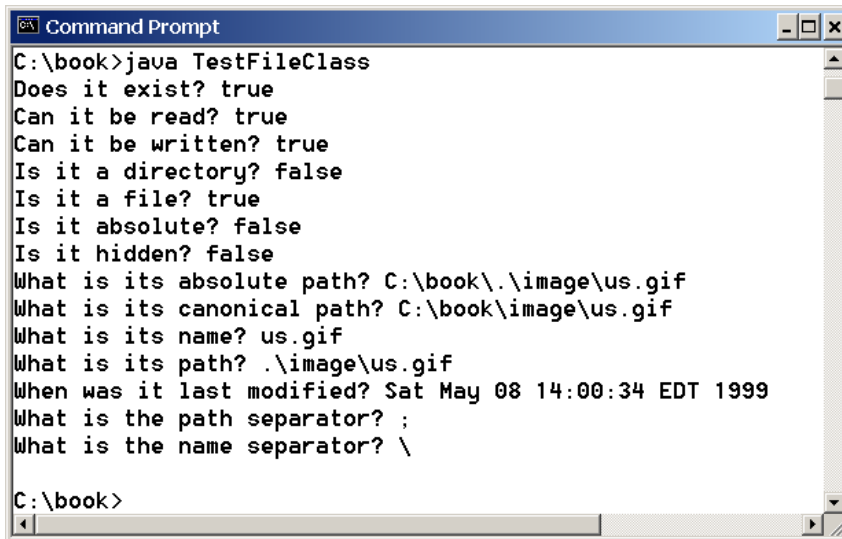
The File class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion. The filename is a string. The File class is a wrapper class for the file name and its directory path.

Obtaining file properties and manipulating file

java.io.File	
+File(pathname: String)	Creates a <code>File</code> object for the specified path name. The path name may be a directory or a file.
+File(parent: String, child: String)	Creates a <code>File</code> object for the child under the directory parent. The child may be a file name or a subdirectory.
+File(parent: File, child: String)	Creates a <code>File</code> object for the child under the directory parent. The parent is a <code>File</code> object. In the preceding constructor, the parent is a string.
+exists(): boolean	Returns true if the file or the directory represented by the <code>File</code> object exists.
+canRead(): boolean	Returns true if the file represented by the <code>File</code> object exists and can be read.
+canWrite(): boolean	Returns true if the file represented by the <code>File</code> object exists and can be written.
+isDirectory(): boolean	Returns true if the <code>File</code> object represents a directory.
+isFile(): boolean	Returns true if the <code>File</code> object represents a file.
+isAbsolute(): boolean	Returns true if the <code>File</code> object is created using an absolute path name.
+isHidden(): boolean	Returns true if the file represented in the <code>File</code> object is hidden. The exact definition of <i>hidden</i> is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
+getAbsolutePath(): String	Returns the complete absolute file or directory name represented by the <code>File</code> object.
+getCanonicalPath(): String	Returns the same as <code>getAbsolutePath()</code> except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
+getName(): String	Returns the last name of the complete directory and file name represented by the <code>File</code> object. For example, new <code>File("c:\\book\\test.dat").getName()</code> returns <code>test.dat</code> .
+getPath(): String	Returns the complete directory and file name represented by the <code>File</code> object. For example, new <code>File("c:\\book\\test.dat").getPath()</code> returns <code>c:\\book\\test.dat</code> .
+getParent(): String	Returns the complete parent directory of the current directory or the file represented by the <code>File</code> object. For example, new <code>File("c:\\book\\test.dat").getParent()</code> returns <code>c:\\book</code> .
+lastModified(): long	Returns the time that the file was last modified.
+length(): long	Returns the size of the file, or 0 if it does not exist or if it is a directory.
+listFile(): File[]	Returns the files under the directory for a directory <code>File</code> object.
+delete(): boolean	Deletes the file or directory represented by this <code>File</code> object. The method returns true if the deletion succeeds.
+renameTo(dest: File): boolean	Renames the file or directory represented by this <code>File</code> object to the specified name represented in <code>dest</code> . The method returns true if the operation succeeds.
+mkdir(): boolean	Creates a directory represented in this <code>File</code> object. Returns true if the the directory is created successfully.
+mkdirs(): boolean	Same as <code>mkdir()</code> except that it creates directory along with its parent directories if the parent directories do not exist.

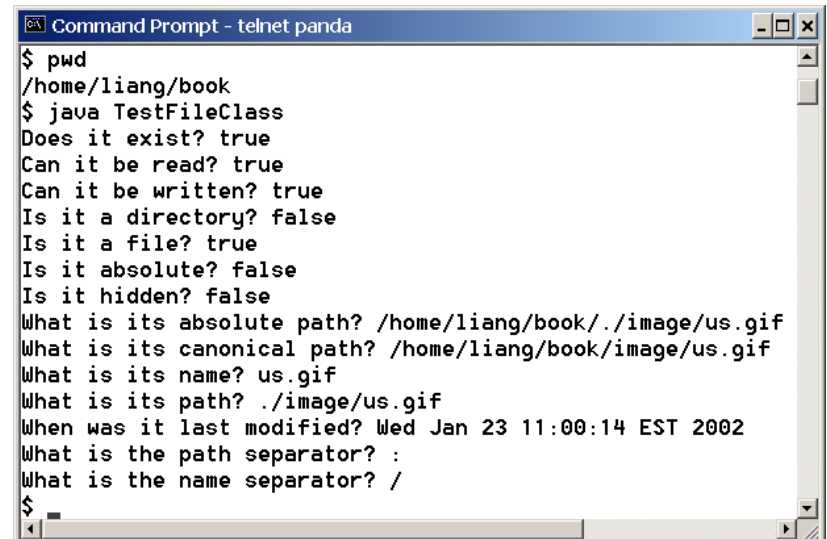
Problem: Explore File Properties

Objective: Write a program that demonstrates how to create files in a platform-independent way and use the methods in the File class to obtain their properties. The following figures show a sample run of the program on Windows and on Unix.



```
Command Prompt
C:\book>java TestFileClass
Does it exist? true
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
What is its absolute path? C:\book\image\us.gif
What is its canonical path? C:\book\image\us.gif
What is its name? us.gif
What is its path? image\us.gif
When was it last modified? Sat May 08 14:00:34 EDT 1999
What is the path separator? ;
What is the name separator? \

C:\book>
```



```
Command Prompt - telnet panda
$ pwd
/home/liang/book
$ java TestFileClass
Does it exist? true
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
What is its absolute path? /home/liang/book/image/us.gif
What is its canonical path? /home/liang/book/image/us.gif
What is its name? us.gif
What is its path? image/us.gif
When was it last modified? Wed Jan 23 11:00:14 EST 2002
What is the path separator? :
What is the name separator? /

$
```

TestFileClass

Text I/O

A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the Scanner and PrintWriter classes.

Writing Data Using PrintWriter

java.io.PrintWriter

+PrintWriter(filename: String)

Creates a PrintWriter for the specified file.

+print(s: String): void

Writes a string.

+print(c: char): void

Writes a character.

+print(cArray: char[]): void

Writes an array of character.

+print(i: int): void

Writes an int value.

+print(l: long): void

Writes a long value.

+print(f: float): void

Writes a float value.

+print(d: double): void

Writes a double value.

+print(b: boolean): void

Writes a boolean value.

Also contains the overloaded
println methods.

A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is `\r\n` on Windows and `\n` on Unix.

Also contains the overloaded
printf methods.

The printf method was introduced in §4.6, “Formatting Console Output and Strings.”

WriteData

Try-with-resources

Programmers often forget to close the file. JDK 7 provides the followings new try-with-resources syntax that automatically closes the files.

```
try (declare and create resources) {  
    Use the resource to process the file;  
}
```

WriteDataWithAutoClose

Reading Data Using Scanner

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

ReadData

Problem: Replacing Text

Write a class named ReplaceText that replaces a string in a text file with a new string. The filename and strings are passed as command-line arguments as follows:

```
java ReplaceText sourceFile targetFile oldString newString
```

For example, invoking

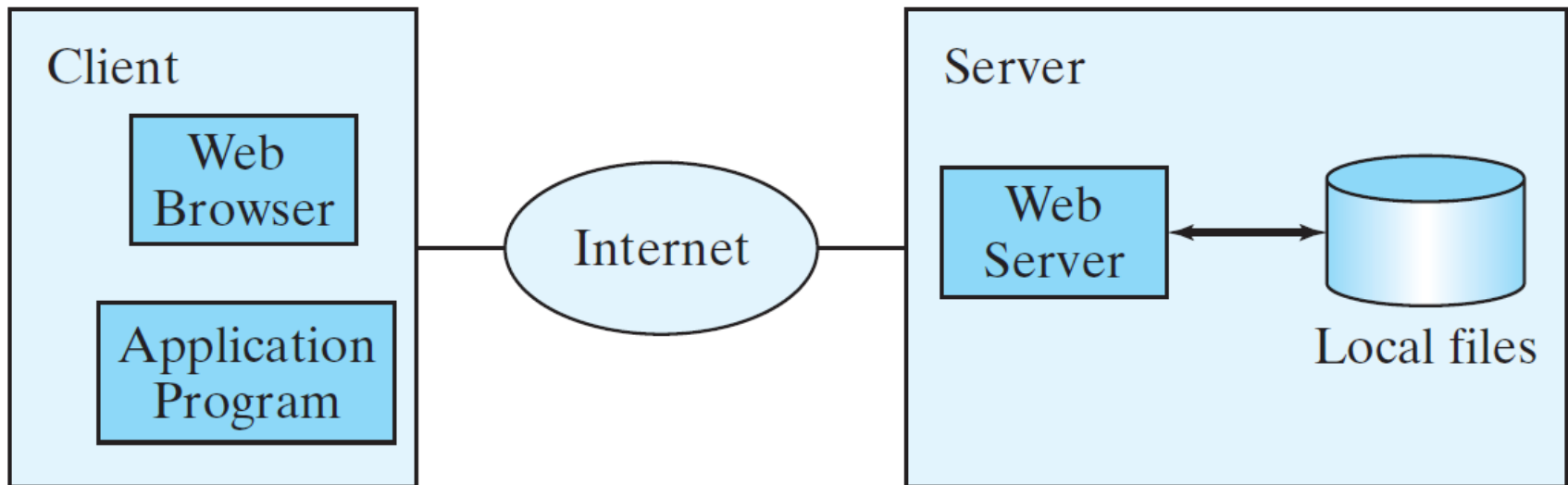
```
java ReplaceText FormatString.java t.txt StringBuilder StringBuffer
```

replaces all the occurrences of StringBuilder by StringBuffer in FormatString.java and saves the new file in t.txt.

ReplaceText

Reading Data from the Web

Just like you can read data from a file on your computer, you can read data from a file on the Web.



Reading Data from the Web

```
URL url = new URL("www.google.com/index.html");
```

After a **URL** object is created, you can use the **openStream()** method defined in the **URL** class to open an input stream and use this stream to create a **Scanner** object as follows:

```
Scanner input = new Scanner(url.openStream());
```

ReadFileFromURL

Thank you
Thank you

